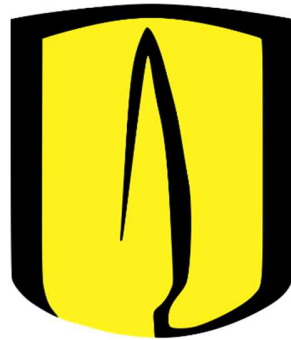


**UNIVERSIDAD DE LOS ANDES DEPARTAMENTO DE INGENIERIA
DE SISTEMAS Y COMPUTACIÓN**



LABORATORIO: INTRODUCCIÓN A REDES DE DATOS

ISIS3204 – INFRAESTRUCTURA DE COMUNICACIONES

Grupo 5

Isabella Lozano - 202221012

Ana Cristina Rodriguez - 202123530

Nicolas Ballen - 202310273

2025-20

Contenido

Introducción.....	3
Configuración Inicial del Sistema	3
Funcionamiento	3
Implementación TCP	4
Implementación UDP	7
Pruebas y Comparación.....	8
Preguntas de Comparación	8
Tabla de Comparación	9
Preguntas de Análisis.....	10
Conclusión.....	15
Bibliografía.....	16

Introducción

El siguiente documento servirá de registro de nuestro desarrollo y evaluación de los sistemas subscriber-publisher con bróker de mensajería que implementamos para evidenciar el comportamiento de los dos protocolos de la capa de transporte: TCP y UDP. A continuación, detallaremos su proceso de configuración, ciertas comparaciones respecto al funcionamiento con cada protocolo, y finalmente la respuesta a algunas preguntas de análisis, las cuales enriquecerán el entendimiento de esta capa del modelo TCP/IP.

Configuración Inicial del Sistema

Para empezar, describiremos la implementación de ambas versiones del sistema. Estas desarrollaron en C, lenguaje con el cual tuvimos acceso a ciertas librerías nativas destinadas al manejo de sockets para la comunicación entre hosts. Específicamente, haciendo uso del Socket API integrado dentro del sistema operativo Linux, dentro del cual se realizó el despliegue de los sistemas y sus respectivas pruebas.

```
#include <arpa/inet.h>
#include <errno.h>
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/select.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <unistd.h>
```

Imagen1: Set de librerías nativas usadas para la implementación, destacando sys.sockets para el manejo de sockets, integrado en C a partir del SocketAPI de Linux

Como sabemos, ambas versiones consistieron en tres archivos .c: el bróker, el suscriptor, y el publicador, que si bien siguieron la misma lógica que veremos más adelante, variaron debido a la lógica propia de cada protocolo.

Funcionamiento

Para propósitos del laboratorio, asumimos un máximo de 5 clientes por rol (10 en total), así como un único tópico al que todos los suscriptores pueden suscribirse. Una vez activo el bróker, este se pone a la espera de nuevos clientes o usuarios en un puerto predeterminado 8080.

```
upo-5/laboratorios/laboratorio3/Laboratorio3/ProtocoloTCP$ ./broker.exe
Broker escuchando en el puerto 8080 ...
```

Imagen 2: Inicialización broker (Ejemplo TCP)

Una vez activo, estará a la espera de la conexión de tanto suscriptores como publicadores, sirviéndonos de una verificación simple del rol mediante un “login” que sirve como mensaje inicial, asignándole un ID de 1 a 10 conforme nuevos clientes se registran:

```

-o-de-Comunicaciones-Grupo-5/laboratorios/laboratorio3/Laboratorio3/ProtocoloTCP$ ./subscriber.exe
Conectado al broker
Login enviado, esperando respuesta...
Login exitoso. Esperando mensajes...

Broker escuchando en el puerto 8080 ...
Nueva conexión aceptada
Cliente agregado a lista de desconocidos en posicion 1
Esperando datos de login del cliente 1
Mensaje recibido: SUBSCRIBER_LOGIN
Cliente registrado como SUBSCRIBER de ID: 1

```

Imagen 3: Ejemplo inicio de sesión de suscriptor

Finalmente, en tanto tengamos suscriptores y publicadores activos, estos últimos enviarán hasta 10 mensajes que el bróker luego retransmitirá a todos los clientes registrados como suscriptores.

Publisher

```

Conectado al broker
Login enviado, esperando respuesta...
Login exitoso. Enviando mensajes periodicamente
Mensaje enviado: PUBLISH: Gol de Equipo A a B
Mensaje enviado: PUBLISH: Gol de Equipo C a D
Mensaje enviado: PUBLISH: Gol de Equipo A a B

```

Subscriber

```

Login enviado, esperando respuesta...
Login exitoso. Esperando mensajes...
^[[A^[[BMensaje recibido: Gol de Equipo A a B
Mensaje recibido: Gol de Equipo C a D
Mensaje recibido: Gol de Equipo A a B
Mensaje recibido: Gol de Equipo C a D
Mensaje recibido: Gol de Equipo A a B
Mensaje recibido: Gol de Equipo C a D
Mensaje recibido: Gol de Equipo A a B
Mensaje recibido: Gol de Equipo C a D
Mensaje recibido: Gol de Equipo A a B
Mensaje recibido: Gol de Equipo C a D

```

Broker

```

Esperando datos de login del cliente 1
Mensaje recibido: PUBLISHER_LOGIN
Cliente registrado como PUBLISHER de ID: 2
Mensaje de Publisher ID: 2: PUBLISH: Gol de Equipo A a B

Mensaje limpio: Gol de Equipo A a B
Mensaje enviado a subscriber 1
Mensaje de Publisher ID: 2 enviado a todos los Subscribers
Mensaje de Publisher ID: 2: PUBLISH: Gol de Equipo C a D

Mensaje limpio: Gol de Equipo C a D
Mensaje enviado a subscriber 1
Mensaje de Publisher ID: 2 enviado a todos los Subscribers

```

Imagen4: Ejemplo Envío y Retransmision de Mensajes (1 publisher y 1 suscriptor)

En el caso del publicador (al que le pusimos el límite de 10 mensajes por las pruebas), este puede enviar un mensaje FIN, tal que el bróker sepa del fin de envíos de su parte y pueda descartarlo, permitiendo así la llegada de otros publicadores.

Implementación TCP

En ambas versiones se manejo el uso de sockets, con la diferencia de tener que definir el tipo de protocolo con el que se comunicarían los mismos. En el caso de TCP, la implementación de C definía estos sockets como del tipo SOCK_STREAM

```

//Socket TCP del broker/servidor
listen_fd = socket(AF_INET, SOCK_STREAM, 0);
if (listen_fd < 0) {
    perror("socket failed");
    exit(EXIT_FAILURE);
}

```

Imagen 5: Ejemplo de inicialización de un socket (Ejemplo Broker TCP)

Ahora bien, la comunicación TCP, basada en conexiones, se regula precisamente mediante los sockets tanto del cliente como del servidor. Por lo mismo, los métodos integrados “send” y “read” de C se sirven de un socket, al cual enviarán o del cual leerán la información que corresponda del host asociado a dicho socket. Si dichas acciones nunca ocurren o registran vacías, se entiende que el host del cual se quería leer o enviar el mensaje se desconectó:

```
valread = read(sd, buffer, BUFFER_SIZE-1);  
bytes_sent=send(client_fd, msg1, strlen(msg1), 0);
```

Imagen 6: Ejemplo Funciones read y send en implementación TCP (siendo sd y client_fd sockets)

Adicionalmente, dado el manejo de bajo nivel de C, se requiere un buffer o espacio donde guardar tanto lo que se envía como lo que se lee, siendo esta la responsabilidad del buffer descrito en la imagen (a lo que se le suma su longitud para saber el tamaño del mensaje que se lee o envía).

- **Broker:**

El bróker sirve como servidor dentro de la relación cliente-servidor que actúa dentro de este sistema tanto en TCP como en UDP, más las funciones o configuraciones previas que necesita para enviar o recibir mensajes varían con cada protocolo. En primer lugar, como servidor con dirección fija, el socket que inicialicemos debe tener configurada dicha dirección antes de esperar cualquier conexión (tal que el cliente tenga idea de a donde conectarse claro):

```
//linqueamos la direccion y el puerto al socket del servidor  
if (bind(listen_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {  
    perror("bind failed");  
    exit(EXIT_FAILURE);  
}  
  
//Ponemos el socket en modo escucha para esperar conexiones entrantes  
if (listen(listen_fd, BACKLOG) < 0) {  
    perror("listen failed");  
    exit(EXIT_FAILURE);  
}
```

Imagen 7: Configuración y enlazamiento de la dirección del servidor al socket (local)

La función “bind()” permite esto como vemos en la imagen, uniendo al socket del bróker “listen_fd” con la dirección y puerto previamente definidos. Adicionalmente, para el caso TCP, el socket se puede poner en modo “escucha” con “listen()”, sumándole un BACKLOG o indicador de cuantas conexiones en cola puede manejar antes de descartar otras conexiones entrantes.

Finalmente, sabemos que el bróker debe manejar múltiples conexiones simultaneas, ante lo cual necesitábamos una implementación que le permitiera actuar sobre el estímulo identificable de cada host. Para ello usamos “select()”.

```

//Variables de select. Basicamente un poll de sockets
fd_set readfds;
int max_sd, sd, activity, valread;
char buffer[BUFFER_SIZE];

// Reseteamos el poll o set de sockets
FD_ZERO(&readfds);
//Agregar el socket del servidor (broker)
FD_SET(listen_fd, &readfds);
max_sd = listen_fd;

//Agregar sockets de los clientes actuales que siguen conectados
for (int i = 0; i < MAX_CLIENTS; i++) {
    sd = clients[i].fd;
    if (sd > 0) FD_SET(sd, &readfds);
    if (sd > max_sd) max_sd = sd;
}

// Esperar accion de un cliente (publisher, subscriber, o nuevo)
activity = select(max_sd + 1, &readfds, NULL, NULL, NULL);

// Nueva conexion entrante (cliente desconocido se conecta al socket del servidor)
if (FD_ISSET(listen_fd, &readfds)) {

```

Imagen 8: Configuración set de sockets, reinicio e inclusión de sockets de clientes previamente registrados, y detección de actividad

Select básicamente retiene la ejecución hasta detectar que uno de los sockets registrados en su set (readfds) presentó alguna acción, siendo la función `FD_ISSET()` la encargada de identificar si dicha acción ocurrió dentro del socket especificado (siempre y cuando este pertenezca al set de sockets indicado). Por ello, cada vez que en el `listen_fd` (socket del bróker) se detecta la llegada de una nueva conexión, la guardamos en su respectivo socket o socket del cliente (función `accept`), los cuales posteriormente se agregaran al set de sockets de `select()` para detectar futuros mensajes enviados por los publicadores registrados.

```

// Nueva conexion entrante (cliente desconocido se conecta al socket del servidor)
if (FD_ISSET(listen_fd, &readfds)) {
    int addrlen = sizeof(address);
    //definimos el socket del cliente
    client_socket = accept(listen_fd, (struct sockaddr *)&address, (socklen_t *)&addrlen);

```

Imagen 9: Uso de función `accept()` para el reconocimiento del socket de un nuevo cliente

Nótese adicionalmente como en la imagen 8 ejecutamos `FD_ZERO` para reiniciar el set de sockets. Esto lo hacemos ya que con cada iteración en la que el bróker tramita un nuevo cliente o mensaje de un publicador. Esto se hace para evitar conservar sockets que pudieran haberse cerrado o perdido por desconexión, de los cuales muy seguramente no se recibirá acción alguna. De ahí que tengamos un arreglo de clientes como se ve en la misma imagen, en la que guardamos los sockets y verificamos si este está activo para regresarlo al set de `select()` (no es -1, que es valor que le definimos al removerlos por desconexión).

De ahí en adelante, el código del bróker consiste en identificar el origen de cualquier nueva acción dentro de los sockets aceptados de los clientes nuevos y/o previamente registrados, eliminando aquellos que se identifiquen como desconectados (lecturas y envíos de 0/no completados), leyendo los logins de nuevos usuarios, mensajes de publicadores, y luego enviando tanto confirmaciones como esos mensajes a los suscriptores respectivamente.

- **Publicador y Suscriptor:**

Tanto el publicador como el suscriptor adoptan el rol de cliente dentro de la relación cliente-servidor de estos sistemas. En el caso de TCP, la diferencia principal con la configuración del bróker radica en que el socket construido no se enlaza o hace “bind()” con la dirección propia del cliente, asumiendo su naturaleza dinámica. Por el contrario, estos sockets de cliente se conectan al socket del servidor o bróker mediante el método “connect()”

```
//Socket TCP del cliente
if ((client_fd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
    perror("Socket creation error");
    return -1;
}
//Definimos la direccion del servidor (local por el momento)
serv_addr.sin_family = AF_INET;
serv_addr.sin_port = htons(PORT);
if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <= 0) {
    perror("Invalid address/ Address not supported");
    return -1;
}
//Conectamos al broker, indicando si hubo error
if ((status=connect(client_fd, (struct sockaddr *)&serv_addr, sizeof(serv_addr))) < 0) {
    perror("Connection failed");
    return -1;
}
printf("Conectado al broker\n");
```

Imagen 10: Configuración conexión cliente a servidor en sockets C (Ejemplo Publisher)

Como vemos en la imagen anterior, una vez creado el bróker y definida la dirección del servidor/bróker, el socket nunca se enlaza a una dirección fija, sino que se conecta a aquel socket con la misma dirección y puerto definidos por parámetros en la función connect. Una vez se confirma la conexión, podemos seguir con el intercambio correspondiente de sends y reads entre ambas partes:

```
buffer[valread] = '\0';
//Si se verifica, empezamos a enviar mensajes periodicamente hasta completar 10 mensajes.
if (strcmp(buffer, "PUBLISH_LOGIN_OK", 16) == 0) {
    printf("Login exitoso. Enviando mensajes periodicamente\n");
    while(mensajes_enviados<10) {
        // Envía el mensaje 1 y 2 con un segundo de espera entre ellos
        //(simulamos eventos deportivos que ocurren en el tiempo, no todos juntos)
        bytes_sent=send(client_fd, msg1, strlen(msg1), 0);
        printf("Mensaje enviado: %s", msg1);
        if (bytes_sent <= 0) {
            perror("Error enviando mensaje 1");
            mensajes_fallidos++;
        }
        sleep(1);
        bytes_sent=send(client_fd, msg2, strlen(msg2), 0);
        printf("Mensaje enviado: %s", msg2);
        if (bytes_sent <= 0) {
            perror("Error enviando mensaje 2");
            mensajes_fallidos++;
        }
        sleep(1);
        mensajes_enviados+=2;
    }
}
```

Imagen 11: Ejemplo interacción del lado del Publisher post-login. Reconoce la confirmación mediante read(), para luego enviar periódicamente con send() los mensajes mock predefinidos.

Implementación UDP

Los patrones de comunicación se conservan en la implementación de UDP. Sin embargo, dado que este protocolo no se orienta bajo una conexión, métodos como “connect()” del lado del cliente o “accept()” de lado del servidor no funcionan. De hecho, la estructura de nuestros clientes cambia al no poder recuperar un socket de su lado en el bróker, siendo que recibe indiscriminadamente los datagramas de todos los orígenes. En pocas palabras, el bróker ahora se sirve únicamente de su propio socket, por lo cual usamos la dirección asociada a los

datagramas recibidos como referencia para distinguir a los clientes y así poder responder acorde:

```
// Estructura para manejar cliente TCP
typedef struct {
    int fd;
    client_role_t role;
} client_t;

// Estructura para manejar cliente UDP
typedef struct {
    struct sockaddr_in addr;
    client_role_t role;
    //Suple la funcion de socket
    int in_use;
} udp_client_t;
```

Imagen 12: Estructuras de cliente TCP (izquierda) y UDP (derecha)

Adicionalmente, las funciones de send() y read() son reemplazadas por sendto() y readto() para UDP, las cuales requieren adicional al socket la dirección de destino.

```
//Recibimos mensaje
valread = recvfrom(listen_fd, buffer, BUFFER_SIZE-1, 0, (struct sockaddr *)&cli, &clen);
sprintf(msg, "PUBLISH_LOGIN_OK\n");
sendto(listen_fd, msg, strlen(msg), 0, (struct sockaddr *)&cli, clen);
```

Imagen 13: Ejemplo uso recvfrom y sendto desde el Broker

Consecuentemente, el bróker no tiene como detectar la desconexión ni los clientes si el bróker se desactivo, en tanto no hay conexión sino direcciones a las que apuntar sin espera de una confirmación o 3wayhandshake que asegure el paso/constancia de información.

Pruebas y Comparación

Preguntas de Comparación

En pos de comparar el funcionamiento de ambos protocolos, hicimos la captura del tráfico de paquetes en ambos sistemas, con lo cual respondimos las siguientes preguntas.

- **En TCP: ¿los mensajes llegan completos y en orden? ¿Cómo maneja TCP la confiabilidad y el control de flujo?**

Los mensajes efectivamente llegaron completos y en orden, alternando entre el mensaje uno (A contra B) y el mensaje 2 (C contra D). En cuanto a la confiabilidad, en wireshark podemos ver el manejo del 3WayHandShake previo al intercambio de datos con una nueva conexión:

No.	Time	Source	Destination	Protocol	Length	Info
55	6.933189	127.0.0.1	127.0.0.1	TCP	7	34762 → 8080 [SYN] Seq=0 Win=65495 Len=0 MSS=65535
56	6.933260	127.0.0.1	127.0.0.1	TCP	7	8080 → 34762 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0
57	6.933275	127.0.0.1	127.0.0.1	TCP	6	34762 → 8080 [ACK] Seq=1 Ack=1 Win=65536 Len=0

Imagen 14: Ejemplo Capturado del 3WayHandshake Previo al Envío del Login de un Publisher Nuevo

Posterior a ello, el uso de los números de secuencia se mantiene para el envío de nuevos mensajes, junto con un intercambio de ACKs que confirman la llegada de los paquetes, así

como un tamaño de ventana “Window” que sirve de control de flujo al regular el envío en función de la capacidad de recepción del bróker.

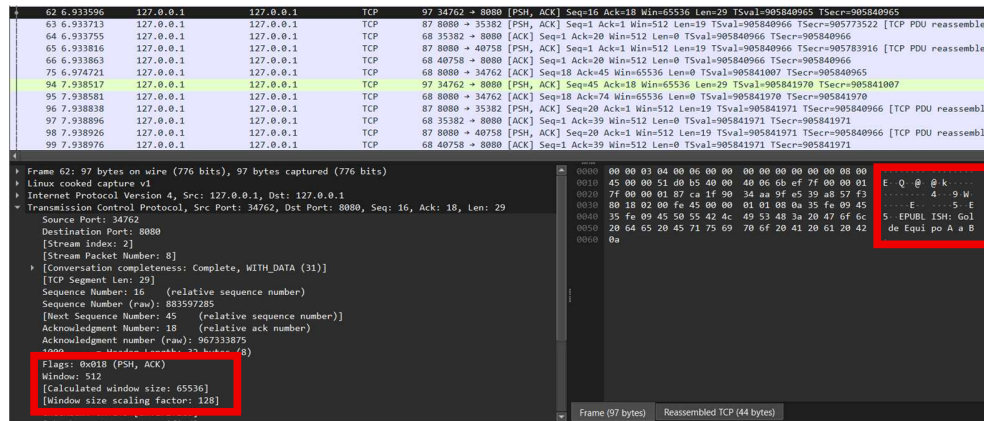


Imagen 15: Envío capturado de mensaje limpio del Publisher, con ventana calculada

• En UDP: ¿qué evidencias hay de pérdida o desorden en los mensajes?

Debido a lo pequeño de los mensajes y la reducida frecuencia con la que se enviaban (2 publishers), no pudimos evidenciar directamente la pérdida o desorden de paquetes. Sin embargo, algunos mensajes pudieron haberse recibido en desorden. Aun así, no fue evidente por lo liviano de los mensajes, aun enviados en simultaneo.

• ¿Qué diferencias observa en el manejo de la conexión entre ambos protocolos?

Adicional a las diferencias de contenido entre UDP, vemos que el envío de estos últimos es mucho más rápido y liviano, siendo que no requerimos del establecimiento/verificación de una conexión para empezar a enviar información. El registro de un nuevo Publisher, por ejemplo, solo le toma a UDP dos intercambios (el envío del login y la respuesta de confirmación), mientras que a TCP le toma esto junto con todo el 3wayhandshake para confirmar la conexión del nuevo Publisher, incrementando lo intercambios (que dado el header de TCP son más pesados) por hasta 3 veces como vemos en la imagen:

No.	Time	Source	Destination	Protocol	Length	Info
48	6.453576	127.0.0.1	127.0.0.1	UDP	59	45797 → 8080 Len=15
49	6.453699	127.0.0.1	127.0.0.1	UDP	61	8080 → 45797 Len=17

No.	Time	Source	Destination	Protocol	Length	Info
55	6.933189	127.0.0.1	127.0.0.1	TCP	76	34762 → 8080 [SYN] Seq=0 Min=5495 Len=0 MSS=65495 SACK_PERM TSval=905840965 TSecr=0 WS=128
56	6.933268	127.0.0.1	127.0.0.1	TCP	76	8080 → 34762 [SYN, ACK] Seq=0 Ack=1 Min=5493 Len=0 MSS=65495 SACK_PERM TSval=905840965 TSecr=905840965 WS=128
57	6.933275	127.0.0.1	127.0.0.1	TCP	68	34762 → 8080 [ACK] Seq=1 Ack=1 Min=5536 Len=0 TSval=905840965 TSecr=905840965
58	6.933425	127.0.0.1	127.0.0.1	TCP	83	34762 → 8080 [PSH, ACK] Seq=1 Ack=1 Min=5536 Len=15 TSval=905840965 TSecr=905840965 [TCP PDU reassembled in 62]
59	6.933436	127.0.0.1	127.0.0.1	TCP	68	8080 → 34762 [ACK] Seq=1 Ack=16 Min=5536 Len=0 TSval=905840965 TSecr=905840965
60	6.933469	127.0.0.1	127.0.0.1	TCP	85	8080 → 34762 [PSH, ACK] Seq=1 Ack=18 Min=5536 Len=17 TSval=905840965 TSecr=905840965

Imagen 16: Paquetes de envío de login y respuesta de confirmación para un nuevo Publisher (UDP arriba y TCP abajo)

Tabla de Comparación

Para terminar, y en base a la teoría vista, podemos comparar el desempeño de ambos protocolos en la siguiente tabla:

Protocolo/ Criterio	Confiabilidad	Orden de Entrega	Pérdida de Paquetes	Overhead/ Cabecera
TCP	El uso de secuencias y retransmisión de paquetes asegura la llegada completa u ordenada de los datos. A ello se suma la verificación de conexión o el 3WayHandShake, con lo que el destino está asegurado.	La sección de SEC y ACK aseguran el manejo ordenado de los paquetes, aún si estos no llegan en el orden en que se enviaron.	Mediante la sección ACK del header, el emisor puede identificar la llegada o pérdida de un(os) paquete(s), con lo cual se puede retransmitir las veces necesarias hasta que se reciba dicho paquete(s).	Cabecera considerablemente pesada de entre 20 y 60 bytes que contiene todas las secciones o flags necesarias para el control de flujo y ordenamiento de los paquetes que se envíen. Asegura la confiabilidad, pero hace los paquetes mucho más pesados y los envíos más lentos.
UDP	Implementa métodos de seguridad como el checksum para la integridad, pero de ahí en adelante nada en el protocolo asegura la llegada del paquete a su destino o que una lectura será en algún momento atendida.	Los paquetes se reciben e interpretan en el orden que lleguen, sea este o no el orden con el que se enviaron (no discrimina, solo recibe).	No hay conexión que persista o que revise la completitud de los datos. Es decir, no hay forma de identificar pérdidas, por lo que estas son permanentes.	Headers reducidos de máximo 8 bytes limitado al checksum y los puertos de cada host. Su poca información no permite dar confiabilidad, pero al ser tan ligero y libre de verificaciones, los envíos son mucho más rápidos.

Preguntas de Análisis

1. ¿Qué ocurriría si en lugar de dos publicadores (partidos transmitidos) hubiera cien partidos simultáneos? ¿Cómo impactaría esto en el desempeño del broker bajo TCP y bajo UDP?

TCP: El broker tendría que mantener cien conexiones simultáneas, una por cada publicador y suscriptores. Esto generaría un alto consumo de CPU y memoria, ya que TCP gestiona control de flujo, congestión, retransmisiones y orden de entrega. En definitiva, el desempeño bajaría al aumentar las conexiones.

UDP: Los mensajes se enviarían sin mantener conexión, por lo que el broker soportaría más transmisiones simultáneas con menor carga. Sin embargo, habría mayor riesgo de pérdida o desorden, una de las principales consecuencias del uso de UDP, lo que afectaría la confiabilidad de las actualizaciones.

2. Si un gol se envía como mensaje desde el publicador y un suscriptor no lo recibe en UDP, ¿qué implicaciones tendría para la aplicación real? ¿Por qué TCP maneja mejor este escenario?

Si un gol se envía como mensaje desde el publicador y un suscriptor no lo recibe en UDP, la actualización simplemente se pierde, ya que este protocolo no posee mecanismos de retransmisión ni confirmación de entrega. En una aplicación real, esto significa que el usuario no vería reflejado el gol en su marcador, generando inconsistencias o desinformación frente a otros usuarios. En cuanto a TCP, este maneja mejor este escenario porque garantiza la entrega confiable de los mensajes. Si un paquete se pierde, el protocolo lo detecta mediante la ausencia de confirmaciones (ACK) y realiza retransmisiones automáticas, asegurando que el evento llegue correctamente y en orden al receptor.

3. En un escenario de seguimiento en vivo de partidos, ¿qué protocolo (TCP o UDP) resultaría más adecuado? Justifique con base en los resultados de la práctica.

Para un escenario de seguimiento en vivo de partidos, el protocolo UDP resulta más adecuado, ya que ofrece menor latencia y permite la transmisión continua de datos sin la sobrecarga de confirmaciones y control de errores que tiene TCP. En la práctica se observa que TCP requiere establecimiento de conexión y envía numerosos mensajes de control (SYN, ACK, PSH), lo que resulta en retrasos. Por el otro lado, UDP transmite datagramas pequeños y frecuentes directamente, de esta manera prioriza inmediatez sobre fiabilidad. Dado que en un sistema en vivo es preferible recibir información actualizada aunque se pierdan algunos paquetes, UDP es la mejor opción para mantener la fluidez y rapidez en la actualización de los datos del partido.

No.	Time	Source	Destination	Protocol	Length	Info
55	6.933189	127.0.0.1	127.0.0.1	TCP	76	34762 → 8880 [SYN, ACK] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=905840965 TSecr=905840965 WS=128
56	6.933260	127.0.0.1	127.0.0.1	TCP	76	8880 → 34762 [ACK] Seq=1 Win=65536 Len=0 MSS=65495 SACK_PERM TSval=905840965 TSecr=905840965
57	6.933275	127.0.0.1	127.0.0.1	TCP	68	34762 → 8880 [PSH, ACK] Seq=1 Win=65536 Len=15 TSval=905840965 TSecr=905840965 [TCP PDU reass...
58	6.933425	127.0.0.1	127.0.0.1	TCP	68	8880 → 34762 [ACK] Seq=1 Win=65536 Len=0 TSval=905840965 TSecr=905840965
59	6.933436	127.0.0.1	127.0.0.1	TCP	85	8880 → 34762 [PSH, ACK] Seq=1 Win=65536 Len=17 TSval=905840965 TSecr=905840965
60	6.933469	127.0.0.1	127.0.0.1	TCP	68	34762 → 8880 [ACK] Seq=16 Ack=18 Win=65536 Len=0 TSval=905840965 TSecr=905840965
61	6.933532	127.0.0.1	127.0.0.1	TCP	68	34762 → 8880 [PSH, ACK] Seq=16 Ack=18 Win=65536 Len=29 TSval=905840965 TSecr=905840965
62	6.933596	127.0.0.1	127.0.0.1	TCP	87	8880 → 35382 [PSH, ACK] Seq=1 Win=512 Len=19 TSval=905840966 TSecr=905773522 [TCP PDU reassemb...
63	6.933713	127.0.0.1	127.0.0.1	TCP	68	35382 → 8880 [ACK] Seq=1 Win=512 Len=0 TSval=905840966 TSecr=905840966
64	6.933755	127.0.0.1	127.0.0.1	TCP	87	8880 → 48758 [PSH, ACK] Seq=1 Win=512 Len=19 TSval=905840966 TSecr=905783916 [TCP PDU reassemb...
65	6.933816	127.0.0.1	127.0.0.1	TCP	68	48758 → 8880 [ACK] Seq=1 Win=512 Len=0 TSval=905840966 TSecr=905840966
66	6.933863	127.0.0.1	127.0.0.1	TCP	68	8880 → 34762 [ACK] Seq=18 Ack=45 Win=65536 Len=0 TSval=905841007 TSecr=905840965
75	6.974721	127.0.0.1	127.0.0.1	TCP	97	34762 → 8880 [PSH, ACK] Seq=45 Ack=18 Win=65536 Len=29 TSval=905841970 TSecr=905841007
94	7.938517	127.0.0.1	127.0.0.1	TCP	68	8880 → 34762 [ACK] Seq=18 Ack=74 Win=65536 Len=0 TSval=905841970 TSecr=905841970
95	7.938581	127.0.0.1	127.0.0.1	TCP	68	8880 → 35382 [PSH, ACK] Seq=20 Ack=1 Win=512 Len=19 TSval=905841971 TSecr=905840966 [TCP PDU reass...
96	7.938838	127.0.0.1	127.0.0.1	TCP	68	35382 → 8880 [ACK] Seq=1 Win=512 Len=0 TSval=905841971 TSecr=905841971
97	7.938896	127.0.0.1	127.0.0.1	TCP	68	8880 → 48758 [PSH, ACK] Seq=20 Ack=1 Win=512 Len=19 TSval=905841971 TSecr=905840966 [TCP PDU reass...
98	7.938926	127.0.0.1	127.0.0.1	TCP	68	48758 → 8880 [ACK] Seq=1 Win=512 Len=0 TSval=905841971 TSecr=905841971
99	7.938976	127.0.0.1	127.0.0.1	TCP	97	34762 → 8880 [PSH, ACK] Seq=74 Ack=18 Win=65536 Len=29 TSval=905842971 TSecr=905841970
116	8.938861	127.0.0.1	127.0.0.1	TCP	68	8880 → 34762 [ACK] Seq=18 Ack=103 Win=65536 Len=0 TSval=905842971 TSecr=905842971
117	8.938927	127.0.0.1	127.0.0.1	TCP	87	8880 → 35382 [PSH, ACK] Seq=30 Ack=1 Win=512 Len=19 TSval=905842971 TSecr=905841971 [TCP PDU reass...
118	8.939045	127.0.0.1	127.0.0.1	TCP	87	8880 → 35382 [PSH, ACK] Seq=30 Ack=1 Win=512 Len=19 TSval=905842971 TSecr=905841971 [TCP PDU reass...

Imagen 17: La imagen muestra una captura de tráfico al utilizar el protocolo TCP. En esta se evidencian los pasos necesarios para establecer la conexión (SYN, SYN-ACK, ACK) y lo necesario para hacer la transferencia de datos

(PSH, ACK). De esta forma se muestra la entrega ordenada y confiable de TCP, aunque este proceso tenga mayor sobrecarga y latencia.

4. Compare el overhead observado en las capturas Wireshark entre TCP y UDP. ¿Cuál protocolo introduce más cabeceras por mensaje? ¿Cómo influye esto en la eficiencia?

En las capturas de Wireshark se observa que TCP introduce más overhead que UDP, ya que cada segmento TCP incluye cabeceras adicionales necesarias para garantizar la conexión y la entrega confiable de datos (campos como número de secuencia, acuse de recibo, flags de control, ventana, etc.). Además, en TCP se generan paquetes extra para el establecimiento y finalización de la conexión (SYN, ACK, FIN), lo que no ocurre en UDP. En cambio, los datagramas UDP tienen una cabecera mucho más pequeña y no requieren confirmaciones ni control de flujo, por lo que los mensajes se envían directamente con menor tamaño y menor retraso. Esto hace que UDP sea más eficiente en transmisión rápida y continua de datos, mientras que TCP es mucho menos eficiente, pero más fiable y con mejor control de errores.

No.	Time	Source	Destination	Protocol	Length	Info
48	6.453576	127.0.0.1	127.0.0.1	UDP	59	45797 → 8080 Len=15
49	6.453699	127.0.0.1	127.0.0.1	UDP	61	8080 → 45797 Len=17
50	6.453838	127.0.0.1	127.0.0.1	UDP	73	45797 → 8080 Len=29
51	6.453966	127.0.0.1	127.0.0.1	UDP	73	45797 → 8080 Len=29
52	6.453988	127.0.0.1	127.0.0.1	UDP	63	8080 → 55685 Len=19
53	6.454133	127.0.0.1	127.0.0.1	UDP	63	8080 → 43845 Len=19
54	6.454272	127.0.0.1	127.0.0.1	UDP	63	8080 → 55685 Len=19
55	6.454289	127.0.0.1	127.0.0.1	UDP	63	8080 → 43845 Len=19
119	8.459785	127.0.0.1	127.0.0.1	UDP	73	45797 → 8080 Len=29
120	8.459834	127.0.0.1	127.0.0.1	UDP	73	45797 → 8080 Len=29
121	8.459852	127.0.0.1	127.0.0.1	UDP	63	8080 → 55685 Len=19
122	8.459923	127.0.0.1	127.0.0.1	UDP	63	8080 → 43845 Len=19
123	8.460057	127.0.0.1	127.0.0.1	UDP	63	8080 → 55685 Len=19
124	8.460245	127.0.0.1	127.0.0.1	UDP	63	8080 → 43845 Len=19
150	10.346552	127.0.0.1	127.0.0.1	UDP	59	37960 → 8080 Len=15
151	10.346741	127.0.0.1	127.0.0.1	UDP	61	8080 → 37960 Len=17
154	10.346844	127.0.0.1	127.0.0.1	UDP	73	37960 → 8080 Len=29
155	10.346937	127.0.0.1	127.0.0.1	UDP	73	37960 → 8080 Len=29
156	10.346948	127.0.0.1	127.0.0.1	UDP	63	8080 → 55685 Len=19
157	10.347009	127.0.0.1	127.0.0.1	UDP	63	8080 → 43845 Len=19
158	10.347097	127.0.0.1	127.0.0.1	UDP	63	8080 → 55685 Len=19
159	10.347187	127.0.0.1	127.0.0.1	UDP	63	8080 → 43845 Len=19

Imagen 18: Captura de tráfico del protocolo UDP. Se observa que aquí los mensajes se envían directamente sin establecer una conexión previa. A diferencia de TCP, acá se muestran datagramas pequeños enviados de manera continua entre diferentes puertos origen y el puerto destino 8080. No aparecen paquetes de control como SYN o ACK, ni retransmisiones, lo que evidencia que UDP no garantiza la entrega ni el orden de los mensajes.

No.	Time	Source	Destination	Protocol	Length	Info
55	6.933189	127.0.0.1	127.0.0.1	TCP	76	34762 → 8080 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=905840965 TSecr=0 WS=128
56	6.933260	127.0.0.1	127.0.0.1	TCP	76	8080 → 34762 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=905840965 TSecr=905840965
57	6.933275	127.0.0.1	127.0.0.1	TCP	68	34762 → 8080 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=905840965 TSecr=905840965
58	6.933425	127.0.0.1	127.0.0.1	TCP	83	34762 → 8080 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=15 TSval=905840965 TSecr=905840965 [TCP PDU reassemb..]
59	6.933436	127.0.0.1	127.0.0.1	TCP	68	8080 → 34762 [ACK] Seq=1 Ack=16 Win=65536 Len=0 TSval=905840965 TSecr=905840965
60	6.933469	127.0.0.1	127.0.0.1	TCP	85	8080 → 34762 [PSH, ACK] Seq=1 Ack=16 Win=65536 Len=17 TSval=905840965 TSecr=905840965
61	6.933532	127.0.0.1	127.0.0.1	TCP	68	34762 → 8080 [ACK] Seq=16 Ack=18 Win=65536 Len=0 TSval=905840965 TSecr=905840965
62	6.933596	127.0.0.1	127.0.0.1	TCP	97	34762 → 8080 [PSH, ACK] Seq=16 Ack=18 Win=65536 Len=29 TSval=905840965 TSecr=905840965
63	6.933713	127.0.0.1	127.0.0.1	TCP	87	8080 → 35382 [PSH, ACK] Seq=1 Ack=1 Win=512 Len=19 TSval=905840966 TSecr=905773522 [TCP PDU reassemb..]
64	6.933755	127.0.0.1	127.0.0.1	TCP	68	35382 → 8080 [ACK] Seq=1 Ack=20 Win=512 Len=0 TSval=905840966 TSecr=905840966
65	6.933816	127.0.0.1	127.0.0.1	TCP	87	8080 → 48758 [PSH, ACK] Seq=1 Ack=1 Win=512 Len=19 TSval=905840966 TSecr=905783916 [TCP PDU reassemb..]
66	6.933863	127.0.0.1	127.0.0.1	TCP	68	48758 → 8080 [ACK] Seq=1 Ack=39 Win=512 Len=0 TSval=905840966 TSecr=905840966
75	6.974721	127.0.0.1	127.0.0.1	TCP	68	8080 → 34762 [ACK] Seq=18 Ack=45 Win=65536 Len=0 TSval=905841007 TSecr=905840965
94	7.938517	127.0.0.1	127.0.0.1	TCP	97	34762 → 8080 [PSH, ACK] Seq=45 Ack=18 Win=65536 Len=29 TSval=905841970 TSecr=905841007
95	7.938581	127.0.0.1	127.0.0.1	TCP	68	8080 → 34762 [ACK] Seq=18 Ack=74 Win=65536 Len=0 TSval=905841970 TSecr=905841970
96	7.938838	127.0.0.1	127.0.0.1	TCP	87	8080 → 35382 [PSH, ACK] Seq=20 Ack=1 Win=512 Len=19 TSval=905841971 TSecr=905840966 [TCP PDU reassemb..]
97	7.938896	127.0.0.1	127.0.0.1	TCP	68	35382 → 8080 [ACK] Seq=1 Ack=39 Win=512 Len=0 TSval=905841971 TSecr=905841971
98	7.938926	127.0.0.1	127.0.0.1	TCP	87	8080 → 48758 [PSH, ACK] Seq=20 Ack=1 Win=512 Len=19 TSval=905841971 TSecr=905840966 [TCP PDU reassemb..]
99	7.938976	127.0.0.1	127.0.0.1	TCP	68	48758 → 8080 [ACK] Seq=1 Ack=39 Win=512 Len=0 TSval=905841971 TSecr=905841971
116	8.938861	127.0.0.1	127.0.0.1	TCP	97	34762 → 8080 [PSH, ACK] Seq=74 Ack=18 Win=65536 Len=29 TSval=905842971 TSecr=905841970
117	8.938927	127.0.0.1	127.0.0.1	TCP	68	8080 → 34762 [ACK] Seq=18 Ack=103 Win=65536 Len=0 TSval=905842971 TSecr=905842971
118	8.939045	127.0.0.1	127.0.0.1	TCP	87	8080 → 35382 [PSH, ACK] Seq=39 Ack=1 Win=512 Len=19 TSval=905842971 TSecr=905841971 [TCP PDU reassemb..]

Imagen 19: Captura de TCP (definición igual a la misma de la Imagen 17 mostrada en el punto 3)

- 5. Si el marcador de un partido llega desordenado en UDP (por ejemplo, primero se recibe el 2-1 y luego el 1-1), ¿qué efectos tendría en la experiencia del usuario? ¿Cómo podría solucionarse este problema a nivel de aplicación?**

El usuario vería una información incorrecta o confusa por unos instantes. Esto ocurre porque UDP no garantiza el orden de los paquetes, entregándolos en el orden en que arriban, independientemente de cómo fueron enviados. Para solucionar este problema, la aplicación debe implementar un control adicional a nivel de software, como incluir números de secuencia o marcas de tiempo en los mensajes. Así, el receptor puede reordenar los eventos antes de mostrarlos, manteniendo la coherencia del marcador y compensando la falta de orden que caracteriza al protocolo.

- 6. ¿Cómo cambia el desempeño del sistema cuando aumenta el número de suscriptores interesados en un mismo partido? ¿Qué diferencias se observaron entre TCP y UDP en este aspecto?**

Cuando aumenta el número de suscriptores interesados en un mismo partido, el desempeño del sistema con TCP va empeorando más rápido que con UDP. Esto ocurre porque TCP crea una conexión independiente con cada cliente, manteniendo buffers, confirmaciones ACK y control de flujo por cada suscriptor. A medida que crece el número de conexiones, el servidor debe manejar más sobrecarga de cabeceras, retransmisiones y control de congestión, lo que incrementa la latencia. En cambio, con UDP, el servidor puede enviar un mismo datagrama a varios destinatarios sin mantener estados de conexión, lo que reduce la sobrecarga y permite un mejor escalamiento. En las prácticas se observó que al aumentar los suscriptores, TCP mostró más tráfico de control y retardos mayores, mientras que UDP mantuvo un flujo constante y estable, aunque con posibles pérdidas de paquetes individuales.

- 7. ¿Qué sucede si el broker se detiene inesperadamente? ¿Qué diferencias hay entre TCP y UDP en la capacidad de recuperación de la sesión?**

En TCP, las conexiones se interrumpen y los clientes detectan la pérdida de la sesión cuando no reciben confirmaciones o cuando el enlace se cierra abruptamente, lo que permite implementar mecanismos de reconexión automática. En UDP, no existe una sesión establecida ni detección automática de fallos, los suscriptores simplemente dejan de recibir mensajes y no tienen forma inmediata de saber si el problema está en el broker o en la red.

- 8. ¿Cómo garantizar que todos los suscriptores reciban en el mismo instante las actualizaciones críticas (por ejemplo, un gol)? ¿Qué protocolo facilita mejor esta sincronización y por qué?**

Para garantizar que todos los suscriptores reciban las actualizaciones críticas al mismo tiempo, el protocolo que mejor facilita la sincronización es UDP. Este permite enviar el mismo mensaje simultáneamente a múltiples destinos mediante difusión (*broadcast*) o *multicast*, reduciendo la diferencia de tiempo entre receptores. TCP, por su naturaleza orientada a conexión, envía los datos de manera secuencial a cada cliente y depende de confirmaciones individuales, lo que genera variaciones en los tiempos de entrega. Aunque UDP no garantiza que todos los receptores obtengan el mensaje, su velocidad y capacidad de distribución simultánea lo hacen más adecuado para aplicaciones en vivo donde la sincronización es prioritaria.

- 9. Analice el uso de CPU y memoria en el broker cuando maneja múltiples conexiones TCP frente al manejo de datagramas UDP. ¿Qué diferencias encontró?**

Al analizar el uso de CPU y memoria en el broker, se observa que TCP consume más recursos que UDP cuando maneja múltiples conexiones simultáneas. Esto pasa porque cada conexión TCP requiere mantener un estado interno lo que incrementa tanto la memoria utilizada como la carga de procesamiento del servidor. En cambio, con UDP el broker no necesita establecer ni mantener conexiones, solo recibe y envía datagramas, lo que reduce significativamente la carga de CPU y el uso de memoria. En la práctica se comprobó que al aumentar el número de clientes, el broker con TCP mostraba un incremento notable en el uso de CPU y RAM, mientras que con UDP el consumo se mantenía más estable y bajo, evidenciando que UDP es más liviano y escalable para aplicaciones de transmisión en tiempo real, aunque menos confiable.

- 10. Si tuviera que diseñar un sistema real de transmisión de actualizaciones de partidos de fútbol para millones de usuarios, ¿elegiría TCP, UDP o una combinación de ambos? Justifique con base en lo observado en el laboratorio**

Para un sistema real de transmisión de actualizaciones de partidos de fútbol a millones de usuarios, se elegiría una combinación de UDP y TCP. De esta forma es posible aprovechar las ventajas de cada uno según el tipo de información. Con base en lo observado en el laboratorio, UDP es ideal para enviar las actualizaciones en tiempo real, ya que permite una distribución rápida y eficiente con bajo consumo de CPU y memoria, y tolera pequeñas pérdidas sin afectar la experiencia del usuario. Sin embargo, para tareas que requieren

fiabilidad y control, como la autenticación de usuarios, suscripción a partidos o envío de configuraciones, se usaría TCP, garantizando que esos datos críticos lleguen completos y en orden. Esta combinación permitiría mantener la velocidad y escalabilidad de UDP para el flujo de información, y la seguridad y confiabilidad de TCP para el control del sistema, logrando así un desempeño óptimo en escenarios de millones de usuarios.

Conclusión

Con este laboratorio logramos entender de manera práctica el comportamiento real de los protocolos TCP y UDP dentro de la capa de transporte, más allá de la teoría. A través de la implementación del modelo publicador–suscriptor con un bróker central, se pudo observar claramente cómo cada protocolo maneja la comunicación entre procesos y los efectos que esto tiene en el desempeño general del sistema.

En TCP, la comunicación resultó más estable y confiable gracias al control de flujo, la verificación de entrega y la retransmisión de datos perdidos. Sin embargo, todo este control implica una mayor cantidad de cabeceras y mensajes de confirmación, lo que se traduce en más consumo de recursos y una transmisión más lenta. En contraste, UDP permitió un flujo de mensajes mucho más rápido y ligero, sacrificando confiabilidad, pero mejorando la inmediatez, lo cual lo hace ideal para aplicaciones en tiempo real como el envío de actualizaciones o transmisiones en vivo.

En conjunto, la experiencia permitió comprender que ambos protocolos responden a necesidades diferentes. TCP garantiza que los datos lleguen completos y en orden, mientras que UDP prioriza la rapidez y la eficiencia. Por esto, dependiendo del tipo de aplicación, puede ser más útil usar uno u otro, o incluso combinarlos para aprovechar lo mejor de ambos: la confiabilidad de TCP para el control y la flexibilidad y velocidad de UDP para la transmisión continua.

Bibliografía

- Kurose, J. y Ross, K. (2016). Computer Networking: A Top-Down Approach. Addison-Wesley, 7ª edición.
- Donoso Meisel, Y. E. (2025). Presentación de clase: Capa de Transporte. Curso ISIS-3204 Infraestructura de Comunicaciones, Universidad de los Andes.
- Quiroga, N. (2025). Laboratorio 3 – Análisis Capa de Transporte y Sockets. Departamento de Ingeniería de Sistemas y Computación, Universidad de los Andes.
- Socket Programming in C. GeeksforGeeks. Disponible en: <https://www.geeksforgeeks.org/socket-programming-in-c/>
- Socket Programming in C/C++: Handling Multiple Clients on Server without Multithreading. GeeksforGeeks. Disponible en: <https://www.geeksforgeeks.org/socket-programming-in-c-c-handling-multiple-clients-on-server-without-multi-threading/>